

From Zero to Four SaaS Products

When I started building my first SaaS product, I didn't really know what I was doing. I knew how to write code. I had a solid background in AI and Machine Learning. But "SaaS architecture"? Multi-portal hierarchy? Role-based access? These were just buzzwords I'd read about online.

Now I've built four full-stack platforms: an L&D portal, an AI-powered LMS called Saksham, a data management platform (QBER), and an AI recruitment intelligence platform, and I'm writing this because the journey genuinely transformed how I think about software.

1. My First Shock: SaaS Is About Layers, Not Just Pages

When I built my very first portal, I thought of it simply: one login page, one dashboard. Maybe two if I needed an admin.

That's not SaaS. That's a website with a login screen.

The real insight came when I started thinking in hierarchies. My first platform — the L&D Portal — shipped with a 5-tier role architecture: Super Admin → Admin → Mentor → Coach → Learner. Every single role had its own dedicated, locked dashboard. A Coach couldn't see what a Super Admin sees. A Learner couldn't stumble into Admin territory.

This wasn't complexity for complexity's sake. It was an intentional design. Each role gets exactly the tools they need, nothing more, nothing less. That principle became a design mantra I carried into every product after it.

2. Confidence Comes From Shipping, Not Reading

Before I started, I'd consumed plenty of documentation, tutorials, and YouTube videos about SaaS architecture. But nothing, genuinely nothing, built confidence the way completing a product did.

After the L&D Portal, I understood cascade user management: one admin action can create, assign, and manage an entire sub-hierarchy. After Saksham, I understood multi-tenancy and how to isolate company-level data properly. After QBER, I got comfortable with module-based access control and dynamic user limit enforcement. After the Recruitment Platform, I could architect a 4-tier system from scratch and know exactly where the seams would be.

Four products mean four times you pushed through the part where you weren't sure it would work. That repetition is irreplaceable.

3. My AIML Background Wasn't a Detour - It Was the Superpower

Here's something I didn't expect: coming from an AI/ML background turned out to be an enormous advantage in SaaS development, not a distraction from it.

In Saksham, instead of writing rigid rubrics for quiz scoring, I plugged in Google Gemini 2.5 Flash-Lite to evaluate open-ended learner answers across semantic understanding, concept coverage, and answer depth — things a rule-based system could never handle well. The accuracy jumped. Facilitators loved it.

In the Recruitment Platform, I built a dual-engine CV analysis system: Gemini handles the deep semantic matching against job descriptions, while a parallel rule-based engine runs 18 deterministic checks on formatting, readability, and professionalism. Neither engine alone would've been enough. Together, they eliminated manual CV screening entirely.

Even the embedded AI chatbot I built into every Saksham portal, a feature most platforms charge enterprise pricing for, came naturally because I already understood how to prompt and chain LLM responses.

4. Security Is Infrastructure, Not an Afterthought

Early on, I thought of authentication as just "the login part." Build it once, move on.

By the time I built Saksham, I had JWT-secured APIs, bcrypt-hashed passwords, and OTP-based password recovery via SMTP email. Not because I wanted to gold-plate the project, but because I'd seen, through iteration, how every security hole creates downstream problems across every role in the hierarchy.

When you have 6 user roles and multi-tenant data isolation, a poorly secured API isn't just one vulnerability. It's a skeleton key. That realisation made me treat security as a first-class architectural concern, not a layer painted on at the end.

5. Great UX Comes From Role Empathy, Not Just Good Design

One thing that quietly improved across all four products was how I thought about UX. Not visual design but role empathy: the practice of genuinely inhabiting each user type and asking "what does this person actually need right now?"

A Learner on the L&D portal needs to see their progress, their earned coins, and their next module. They don't need to see support ticket statistics or user management tables. Showing them, those things isn't neutral, it's noise that makes the product feel harder.

A Data Entry user on QBER needs a clean, fast data input workflow. A Viewer user needs rich, filterable dashboards with Excel-style sorting and grouping. Same platform. Entirely different experiences. Both optimised.

6. Features You're Proud Of Are Usually the Ones You Invented Under Constraints

Some of the features I'm proudest of across these four platforms came from constraints, not freedom.

AI-generated PDF certificates in Saksham? That came from not wanting to manually create certificates for every learner at scale. The coin-based rewards engine in the L&D Portal? That came from trying to drive engagement without a gamification library. Tiered subscription enforcement in the Recruitment Platform? That came from needing a real monetisation model, not just a demo product.

Constraints forced creative solutions. And those solutions ended up being the differentiating features — the things that made each platform genuinely useful rather than just technically functional.

Summary

Four SaaS products in, here's what I know:

I started thinking SaaS was about code. It's actually about structure, role hierarchies, data isolation, permission boundaries, and user flows designed around real workflows. Code is just how you express that structure.

I started thinking my AIML background was separate from "real" software engineering. It turned out to be the thing that made my products genuinely smarter than what a pure web developer would build.

And I started thinking confidence was something you needed before you built. It's actually something you earn by building — product by product, mistake by mistake, one developed feature at a time.

If you're standing where I was at the beginning with a skill set, a vague idea, and a lot of uncertainty, my advice is simple: build the thing. The knowledge will come. The confidence will follow.